

# Reviewer Pack — Minimal Local Dimension of Semigroups and Read-Twice BP Comp...

Entry f130baaf1a68 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 06:37:28 UTC

## Minimal Local Dimension of Semigroups and Read-Twice BP Complexity

Entry ID: f130baaf1a68 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 06:37:28 UTC

### 1. Conjecture

**Field A** (mathematical branch): Semigroup Theory **Field B** (complexity object): Branching Program: Read-Twice Complexity

#### Statement:

[‘For every instance of the read-twice branching program complexity, there exists a minimal semigroup associated with its structure such that the local dimension of this semigroup is bounded by a function of the length of the branching program.’, ‘The local dimension of a semigroup  $S$  with respect to an element  $a$  is defined as the minimum number of generators required to generate the subsemigroup generated by the elements in a ball of radius  $r$  centered at  $a$ , for some fixed  $r$ .’, ‘This bound holds with high probability over 30 random seeds.’]

#### Rationale (proposer’s reasoning):

[‘Semigroups provide a natural structure for studying combinatorial properties, and their local dimension has been used to study various problems in algebraic geometry and group theory. By associating semigroups with branching programs, we may uncover new insights into the complexity of computing functions represented by these programs.’, ‘The local dimension could potentially expose the underlying algebraic structure that affects the read-twice complexity of a program, providing a novel angle for proving lower bounds.’, ‘Semigroups are computationally accessible and can be efficiently constructed from branching program structures.’]

**Taxonomy category:** semigroup\_theory\_to\_bp\_complexity (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** 7b483c0ebc7f77fe

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

**Acceptance criterion:**

The conjecture is supported if, across 30 random seeds, the mean local dimension of the associated minimal semigroups for read-twice branching programs of length  $n$  does not exceed a function of  $n$  (e.g.,  $3n/4$ ) with a  $p$ -value  $\leq 0.05$  from an ANOVA test.

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|---------|------------|------------------|------------------|
| RELATIVIZATION | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |
| NATURAL PROOFS | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |
| KARP_LIPTON    | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |

### 4. Novelty audit

**Verdict:** NOVEL against 8 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): - 'minimal local dimension semigroups' AND 'read-twice branching program complexity' - 'semigroup theory' AND 'local dimension read-twice BP' - 'branching program complexity' AND 'semigroup structure minimal local dimension'

**Top relevant hits considered:** - [<http://arxiv.org/abs/1612.01265v5>] Branching trees I: Concatenation and infinite divisibility - [<http://arxiv.org/abs/2010.01573v4>] On the complete metrisability of spaces of contractive semigroups - [<http://arxiv.org/abs/2302.06539v3>] On the minimal faithful degree of Rhodes semisimple semigroups - [<http://arxiv.org/abs/1309.3636v2>] Strongly productive ultrafilters on semigroups - [<http://arxiv.org/abs/1107.4821v3>] Lattice isomorphisms of bisimple monogenic orthodox semigroups - [<http://arxiv.org/abs/0910.2075v1>] Constructing Numerical Semigroups of a Given Genus - [<http://arxiv.org/abs/0911.3482v5>] Complexity of Networks (reprise) - [<http://arxiv.org/abs/2401.14623v1>] Structure in Communication Complexity and Constant-Cost Complexity Classes

### 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only,  $\leq 90$ s wall time per cycle **Execution:** rc=1, elapsed=0.1s

#### 5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_branching_program(n):
    program = []
    for _ in range(n):
        if random.choice([True, False]):
            program.append('0')
        else:
            program.append('1')
    return program

def construct_semigroup(program):
    semigroup = set()
    current_state = 0
```

```

for symbol in program:
    if symbol == '0':
        current_state = (current_state * 2) % len(program)
    else:
        current_state = (current_state * 3 + 1) % len(program)
    semigroup.add(current_state)
return semigroup

def local_dimension(semigroup, a, r):
    generators = set()
    for element in semigroup:
        if element not in generators and all(element[i] == generator[i] for i in range(len(generator))):
            generators.add(element)
    return len(generators)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    metric_values = []

    for n in n_values:
        program = generate_branching_program(n)
        semigroup = construct_semigroup(program)
        a = random.choice(list(semigroup))
        r = min(2, len(a)) # Ensure r is at least 1 and not larger than the length of a
        dimension = local_dimension(semigroup, a, r)
        metric_values.append(dimension)

    mean_value = sum(metric_values) / len(metric_values)
    std_value = math.sqrt(sum((x - mean_value) ** 2 for x in metric_values) / len(metric_values))
    conjecture_holds = all(value <= (3 * n / 4) for value, n in zip(metric_values, n_values))

    return {
        "metric_name": "local_dimension",
        "metric_value": mean_value,
        "instances_tested": len(n_values),
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else "mapping_undefined"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")

```

```

elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next((seed for seed, result in enumerate(results) if not result["conjecture"]), None)
    print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cd4a492b.py", line 76, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cd4a492b.py", line 54, in run_trial
    r = min(2, len(a)) # Ensure r is at least 1 and not larger than the length of a
           ^^^^^
TypeError: object of type 'int' has no len()

```

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test code crashed before producing data, which prevents us from evaluating whether the conjecture is supported or falsified. | next: Investigate and fix the error in the test code to allow for proper evaluation of the conjecture.

## 11. Audit log (LLM calls)

**Total LLM calls:** 10

| #  | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|----|-----------------|---------------|------------------|-----------|-----|--------------|
| 1  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 16428        |
| 2  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 14642        |
| 3  | preregistration | ollama_remote | glm4:latest      | 0         | 0   | 9994         |
| 4  | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 8316         |
| 5  | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 9195         |
| 6  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 12205        |
| 7  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 8144         |
| 8  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 9964         |
| 9  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 9195         |
| 10 | judge           | ollama_remote | glm4:latest      | 0         | 0   | 11643        |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 109725 ms total latency. Provider mix: {'ollama\_remote': 10}

*(full prompt+response transcripts available in research/audit/f130baaf1a68.jsonl)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/f130baaf1a68.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** research/pvsnp\_sandbox/test\_\*.py (per-cycle)

**Replay tarball:** research/replay/f130baaf1a68.tar.gz (if generated)